

ANL Staking Protocol — weryfikacja audit4-v1.00

Zakres i ograniczenia

Zweryfikowano bezpośrednio zawartość paczki:

```
anl-protocol-audit4-v1.00.zip
```

Deklarowany HEAD:

```
ddf4b36607180faff8dbe48c3133e27a6b32d036
```

Dowód:

- `AUDIT-HEAD.txt:1-13`

Paczka nie zawiera katalogu `.git`, więc nie mogłem niezależnie wykonać `git show`, porównać historii ani potwierdzić, że pliki rzeczywiście pochodzą z tego commita. W środowisku nie ma również `cargo`, `rustc` ani `anchor`, dlatego nie odtworzyłem testów i buildów dynamicznie.

Najważniejsze zastrzeżenie: zawartość paczki nie odpowiada w pełni deklaracji dotyczącej aktualnego evidence run. `docs/TEST-LOG.txt` jest starym, nieudanym logiem z commita `27cd983`, a plik `docs/TEST-LOG.sha256` w ogóle nie znajduje się w paczce.

1. Weryfikacja punktów 1–4

1.1 `scripts/audit-evidence.sh`

Status: kod skryptu spełnia poprzednie zalecenia, ale paczka nie zawiera jego udanego wyniku

Skrypt został rzeczywiście przepisany fail-closed.

Potwierdzone poprawki

Ma:

```
set -euo pipefail
```

- `scripts/audit-evidence.sh:7`

Przed utworzeniem logu sprawdza całe drzewo, w tym staged i untracked:

```
if [ -n "$(git status --porcelain)" ]; then
    exit 1
fi
```

- `scripts/audit-evidence.sh:10-16`

Log powstaje w `/tmp`, poza repo:

- `scripts/audit-evidence.sh:18-20`

Funkcja `run` używa potoku z `tee`, a `pipefail` sprawia, że błąd właściwego polecenia nie zostaje zamaskowany przez poprawne zakończenie `tee`:

```
"$@" 2>&1 | tee -a "$LOG"
```

- `scripts/audit-evidence.sh:22-26`

Testy negatywne wymagają jednocześnie:

- niezerowego kodu wyjścia;
- dokładnego komunikatu błędu.
- `scripts/audit-evidence.sh:28-45`

Skrypt wykonuje również oba warianty testów `core`:

```
( cd core && run cargo test && run cargo test --features test-periods )
```

- `scripts/audit-evidence.sh:61-66`

Na końcu zapisuje HEAD oraz hash logu:

- `scripts/audit-evidence.sh:84-101`

Ocena zachowania `tee` i `pipefail`

Konstrukcja:

```
"$@" 2>&1 | tee -a "$LOG"
```

jest poprawna przy `set -o pipefail`. Kod wyjścia potoku będzie niezerowy, gdy właściwe polecenie zakończy się błędem, nawet jeśli `tee` zakończy się kodem `0`.

Ocena subshella `core`

Funkcja `run` jest dostępna w subshellu utworzonym przez nawiasy, ponieważ jest to subshell bieżącej powłoki, a nie osobne wykonanie `bash -c`.

`set -e` i `pipefail` również pozostają aktywne. Nie znalazłem tutaj luki pozwalającej przejść dalej po błędzie testów `core`.

Drobne uwagi hardeningowe

Brak sprzątnięcia pliku tymczasowego

Po sukcesie `/tmp/anl-test-log.*` pozostaje na dysku. Nie jest to luka bezpieczeństwa, ale można dodać:

```
trap 'rm -f "$LOG"' EXIT
```

Trzeba wtedy osobno zachować log częściowy przy błędzie, jeżeli jest potrzebny diagnostycznie.

Status po biegu jest praktycznie zawsze pusty

`git status --porcelain` jest wykonywany przed skopiowaniem logu do `docs/`.

- `scripts/audit-evidence.sh:84-97`

Dlatego komunikat:

```
git status --porcelain po biegu
```

dowodzi wyłącznie, że testy nie zmodyfikowały śledzonych plików przed publikacją logu. Nie obejmuje końcowego stanu po zapisaniu `TEST-LOG.txt` i `TEST-LOG.sha256`.

To jest akceptowalne, ale opis powinien być precyzyjny.

Krytyczna niespójność paczki

Dołączony `docs/TEST-LOG.txt` nie został wygenerowany przez aktualny skrypt.

Zawiera:

```
HEAD: 27cd983e6152d619cf7c3ae2a245a6aa07cac617
```

- `docs/TEST-LOG.txt:6-9`

Pokazuje brudne drzewo:

```
M docs/TEST-LOG.txt
```

- docs/TEST-LOG.txt:11-13

Pokazuje nieudany `git diff --check`:

- docs/TEST-LOG.txt:15-20

Wszystkie polecenia Cargo kończą się:

```
cargo: command not found
```

- docs/TEST-LOG.txt:22-50

Co gorsza, ten stary skrypt uznał brak Cargo za poprawny negatywny test:

- docs/TEST-LOG.txt:52-59

W paczce nie ma:

```
docs/TEST-LOG.sha256
```

Werdykt punktu 1

Zmiana skryptu: zamknięta statycznie.

Wymaganie dowodowe przed testnetem: jeszcze nieudowodnione przez tę paczkę.

Aby zamknąć je dowodowo, kolejna paczka musi zawierać świeże:

- docs/TEST-LOG.txt ;
- docs/TEST-LOG.sha256 ;

wygenerowane aktualnym skryptem na HEAD `ddf4b36`, bez komunikatów `command not found` i z kompletem udanych poleceń.

1.2 build-mainnet.sh i build-testnet.sh

Status: poprzednie ustalenie zamknięte statycznie

Oba skrypty sprawdzają:

```
test -z "$(git status --porcelain)"
```

w równoważnej konstrukcji `if`.

Dowód:

- `scripts/build-mainnet.sh:5-11`
- `scripts/build-testnet.sh:4-10`

To wykrywa:

- zmiany unstaged;
- zmiany staged;
- pliki untracked.

Oba manifesty zawierają:

- typ release;
- czas;
- HEAD;
- features;
- SHA-256 binarki;
- wersję Rust.

Dowód:

- `scripts/build-mainnet.sh:12-23`
- `scripts/build-testnet.sh:11-22`

Czy bramki da się obejść?

W normalnym procesie developerskim są odpowiednie.

Istnieją jednak klasyczne, lokalne obejścia Git, na przykład:

- `git update-index --assume-unchanged ;`
- `git update-index --skip-worktree ;`
- modyfikacja pliku poza śledzeniem przez Git;
- użycie zmodyfikowanego toolchaina lub programu `anchor` ;
- ręczne zbudowanie i wdrożenie bez skryptu.

To nie są realistyczne luki kontraktu, lecz granice proceduralnej kontroli lokalnego środowiska.

Dla zamkniętego testnetu wystarcza:

- chroniony CI;
- artefakty pobierane z CI;
- porównanie SHA-256 przed deployem;
- zakaz lokalnego deployu.

Dla mainnetu lepszy jest release tworzony wyłącznie przez chroniony runner CI, nie lokalną maszynę operatora.

Pozostała uwaga

Skrypty nie usuwają poprzedniej binarki przed buildem. `set -e` chroni przed użyciem starego artefaktu po jawnej awarii `anchor build`, ale dla większej jednoznaczności można przed buildem usunąć:

```
rm -f target/deploy/anl_staking.so
```

a po buildzie wymagać:

```
test -s "$BIN"
```

Werdykt punktu 2

Zamknięte dla etapu przedtestnetowego.

1.3 `.github/workflows/ci.yml`

Status: poprzednie ustalenia zamknięte statycznie

CI ma cztery wymagane joby:

- `lint`;
- `test`;
- `release-guards`;
- `supply-chain`.

Release guards

Pierwszy guard wymaga:

- niezerowego kodu;
- komunikatu:

```
test-periods cannot be enabled together with network-mainnet
```

- `.github/workflows/ci.yml:46-59`

Drugi guard również wymaga teraz konkretnego komunikatu:

```
select exactly one network feature
```

- `.github/workflows/ci.yml:60-72`

Pozytywne buildy obu wariantów są sprawdzane:

- `.github/workflows/ci.yml:73-76`

Supply-chain

Usunięto `|| true`.

```
run: cargo audit
```

oraz:

```
run: cargo deny check advisories bans sources
```

- `.github/workflows/ci.yml:78-88`

Nowe, nieuwzględnione advisory nadal powinno zablokować CI, o ile nie zostanie dodane do jawnej listy wyjątków.

Ograniczenie weryfikacji

Nie mam dostępu do prywatnego GitHub Actions ani jego logów. Mogę potwierdzić wyłącznie, że konfiguracja jobów jest blokująca w źródle. Nie mogę niezależnie potwierdzić deklaracji „CI jest zielone na `ddf4b36`”.

Werdykt punktu 3

Zamknięte statycznie.

1.4 README EN i PL

Status: poprzednie ustalenie zasadniczo zamknięte

README nie prezentują już gołego `anchor build` jako ścieżki release.

Obecna polityka mówi:

```
scripts/build-testnet.sh
scripts/build-mainnet.sh
```

- `README.md:67-100`
- `README.pl.md:67-100`

Zwykły `anchor build` jest dopuszczony wyłącznie dla lokalnego developmentu, a jego wynik nie może zostać wdrożony:

- `README.md:95-100`
- `README.pl.md:95-100`

Liczba testów matematyki została poprawiona do 24:

- `README.md:87-89`
- `README.pl.md:87-89`

Feature'y sieciowe i oba `compile_error!` są opisane:

- `README.md:72-83`
- `README.pl.md:72-83`

Pozostała historyczna nieścisłość

W dalszej części README nadal widnieje:

Otwarte: #1

- `README.pl.md:123-130`
- analogiczna sekcja w `README.md`

To jest historycznie nieaktualne, ponieważ dokument bezpieczeństwa oznacza R1-01 jako zamknięte przez model epokowy.

Sekcja może pozostać jako historyczna, ale powinna jawnie mówić:

Stan na koniec audytu #1: ustalenie #1 pozostawało otwarte.
Zamknięte następnie przez model checkpointów epokowych – patrz SECURITY-AUDITS.md.

W obecnej postaci czytelnik może uznać, że problem nadal jest otwarty.

Werdykt punktu 4

Zamknięte dla wskazanych ustaleń V-05, z jedną zalecaną korektą historycznej sekcji audytu #1.

2. Supply-chain — ocena ośmiu wyjątków

Źródło zależności w audytowanej paczce

Z `Cargo.lock` wynika:

```
ed25519-dalek 1.0.1
├─ solana-sdk 1.18.26
├─ an1_staking
```

oraz:

```
curve25519-dalek 3.2.1
├─ ed25519-dalek 1.0.1
├─ solana-sdk 1.18.26
├─ an1_staking
```

`solana-sdk` jest wyłącznie `dev-dependency` :

- `programs/an1_staking/Cargo.toml:25-29`

Pozostały stos:

```
ring / rustls-webpki / quinn-proto
├─ quinn
├─ solana-client
├─ solana-banks-server
├─ solana-program-test
├─ an1_staking
```

także pochodzi z `solana-program-test`, który jest `dev-dependency`.

Zatem na podstawie manifestów i lockfile te konkretne wersje nie powinny być linkowane do produkcyjnego artefaktu SBF programu.

2.1 RUSTSEC-2022-0093 — ed25519-dalek

Podatność dotyczy API, w którym prywatna i publiczna część klucza mogą zostać niebezpiecznie połączone; przy użyciu funkcji podpisującej jako oracle możliwe jest odzyskanie klucza prywatnego. Naprawiona jest w `ed25519-dalek >=2`.

Ocena testnetowa

Akceptowalny wyjątek warunkowy.

Uzasadnienie jest poprawne dla artefaktu SBF, ponieważ program nie wykonuje lokalnego podpisywania kluczami prywatnymi.

Nie oznacza to jednak, że advisory jest obojętne dla całego procesu. Stary `solana-sdk` może być używany przez:

- narzędzia testowe;

- skrypty operacyjne;
- klienta bota;
- narzędzia podpisujące transakcje.

Jeżeli prywatny bot korzysta z tego samego stosu SDK i podpisuje realne testnetowe transakcje, należy zweryfikować, czy nigdy nie konstruuje `Keypair` z niezależnie dostarczonych części publicznej i prywatnej.

Dla testnetu o ograniczonej wartości wyjątek jest akceptowalny. Dla mainnetu — nie.

2.2 `RUSTSEC-2024-0344` — `curve25519-dalek`

Podatność dotyczy zmienności czasowej w odejmowaniu scalarów i potencjalnego wycieku sekretów podczas operacji na tajnych wartościach. Naprawiona jest od `4.1.3`.

Ocena testnetowa

Akceptowalny wyjątek warunkowy.

Nie wpływa na deterministyczne wykonanie programu SBF, o ile ta biblioteka rzeczywiście nie trafia do produkcyjnego artefaktu i nie wykonuje operacji na prywatnych scalarach w programie.

Ryzyko pozostaje w narzędziach klienckich podpisujących transakcje. Dla zamkniętego testnetu i krótkiego, terminowego wyjątku jest to akceptowalne.

2.3 `RUSTSEC-2025-0009` — `ring`

Advisory obejmuje między innymi możliwy panic w QUIC header protection przy specjalnie przygotowanym pakiecie oraz panic dla ogromnego pojedynczego bufora AES z włączonym overflow checking. Poprawka znajduje się w `ring >=0.17.12`.

Ocena testnetowa

Akceptowalny dla artefaktu SBF, ale istotny dla klienta sieciowego.

W tym repo zależność pochodzi przez stos QUIC `solana-program-test`, więc nie jest częścią programu on-chain.

Jeżeli jednak bot lub narzędzia deploymentowe korzystają z analogicznego starego stosu QUIC, złośliwy lub przejęty endpoint może powodować DoS klienta.

Dla zamkniętego testnetu nie uznaję tego za blocker, pod warunkiem:

- używania kontrolowanych RPC;
- braku ekspozycji testowych serwerów QUIC na publiczny Internet;
- automatycznego restartu klienta;
- ograniczonej wartości aktywów.

2.4 RUSTSEC-2026-0037 — quinn-proto

RustSec klasyfikuje tę podatność jako high-severity network DoS w endpointach Quinn.

Ocena testnetowa

Najbardziej wymagający wyjątek, lecz nadal akceptowalny dla zamkniętego testnetu.

Nie jest akceptowalny dla publicznie dostępnego, krytycznego serwera QUIC.

Jeżeli dotknięta wersja występuje wyłącznie w `solana-program-test`, ryzyko dotyczy procesu testowego, nie programu SBF.

Jeżeli jednak ta sama wersja jest używana przez działający bot, RPC proxy lub usługę publiczną, wyjątek wymaga osobnego triażu tego komponentu. Sam argument „SBF nie używa QUIC” nie wystarcza do zaakceptowania advisory dla całej infrastruktury.

2.5 RUSTSEC-2026-0185 — quinn-proto

Podatność pozwala zdalnemu peerowi wywołać nieograniczone zużycie pamięci przez fragmenty strumienia z wieloma lukami. Ma CVSS 7.5, atak sieciowy bez uprawnień i wpływ na dostępność. Poprawiona jest w `quinn-proto >=0.11.15`.

Ocena testnetowa

Akceptowalny tylko przy ścisłym ograniczeniu zakresu.

To najsilniejszy wyjątek z całej listy.

Akceptuję go dla zamkniętego testnetu tylko dlatego, że lockfile pokazuje ścieżkę przez `solana-program-test`, czyli zależność testową.

Nie akceptowałbym go bez dodatkowej analizy, gdyby:

- ten sam lockfile był używany do budowy publicznej usługi;
- bot otwierał niezaufane połączenia QUIC;
- testowy banks server był publicznie dostępny;
- proces był krytyczny dla możliwości wyjścia użytkowników.

2.6 RUSTSEC-2026-0098 — rustls-webpki

Dotyczy błędnej obsługi ograniczeń URI w certyfikatach. Eksploatacja wymaga błędnie wystawionego certyfikatu i następuje po poprawnej weryfikacji podpisu.

Ocena testnetowa

Akceptowalny wyjątek.

Nie wpływa na SBF. Dla klienta TLS ryzyko jest ograniczone i wymaga specyficznego PKI z name constraints oraz misissuance.

2.7 RUSTSEC-2026-0099 — rustls-webpki

Dotyczy akceptacji wildcarda wykraczającego poza dozwolone DNS name constraints. Także wymaga błędnie wystawionego certyfikatu.

Ocena testnetowa

Akceptowalny wyjątek.

Dla standardowego publicznego RPC bez prywatnego enterprise PKI używającego name constraints praktyczna ekspozycja jest ograniczona.

2.8 RUSTSEC-2026-0104 — rustls-webpki

Dotyczy osiągalnego paniku podczas parsowania specjalnie przygotowanej listy CRL. Aplikacje, które nie używają CRL, nie są dotknięte.

Ocena testnetowa

Akceptowalny wyjątek, jeżeli stos nie konfiguruje CRL.

Warto potwierdzić w kodzie klienta lub wersji Rustls używanej przez Solanę, czy CRL są w ogóle dostarczane do walidatora certyfikatów.

Łączny werdykt supply-chain

Dla zamkniętego testnetu

Żaden z ośmiu wyjątków nie jest bezwzględnym blockerem zamkniętego testnetu, ponieważ w tej paczce wszystkie wynikają z `dev-dependencies`, głównie `solana-program-test` i `solana-sdk`, a nie z programu SBF.

Największą ostrożność wymagają:

1. `RUSTSEC-2026-0185` ;
2. `RUSTSEC-2026-0037` ;
3. `RUSTSEC-2022-0093` ;
4. `RUSTSEC-2024-0344` .

Ich akceptacja jest ważna wyłącznie dla programu i jego testów. Nie powinna być automatycznie przenoszona na prywatny bot, wallet tooling, deployment host ani usługę RPC.

Dla immutable mainnetu

Wyjątki powinny zostać usunięte przez migrację stosu, zgodnie z obecnym Definition of Done.

Nie rekomenduję permanentnego ignorowania high-severity advisory QUIC ani advisory dotyczących operacji kluczem prywatnym w narzędziach produkcyjnych.

3. Ocena docs/SECURITY-AUDITS.md

Ogólna ocena

Status: rzetelny jako historia wysokiego poziomu, lecz kilka statusów jest zbyt kategoriycznych

Dokument poprawnie opisuje:

- początkową lukę XNT;
- rolę operatora;
- przejście do modelu epokowego;
- wykrycie brakującego version check;
- owner check;
- H-01;
- V-01–V-05;
- residual rozwodnienia;
- potrzebę fuzzingu;
- blokery immutable mainnetu.

Potwierdzone zgodności z przebiegiem rund

R1-01 jest poprawnie opisane jako pierwotny problem zależny od czasu fundingu:

- docs/SECURITY-AUDITS.md: 19–29

Dokument odnotowuje, że po pierwszych poprawkach ustalenie #1 pozostało otwarte:

- docs/SECURITY-AUDITS.md: 41–44

Opis modelu:

```
end_epoch = epoch_of(end_ts - 1)
```

jest zgodny z późniejszą specyfikacją:

- docs/SECURITY-AUDITS.md: 53–61

Runda #3 poprawnie wymienia:

- M-01;
- M-02;
- L-01;
- H-01.

- docs/SECURITY-AUDITS.md:67-76

Dwie późniejsze weryfikacje oraz V-01–V-05 są odnotowane:

- docs/SECURITY-AUDITS.md:79-99

Problemy rzetelności

[DOC-01] „version checks in every instruction context” jest zbyt szerokie

Tabela mówi:

```
R1-07 – version == ACCOUNT_VERSION in every instruction context
```

- docs/SECURITY-AUDITS.md:117

W poprzedniej rundzie ustaliliśmy, że `UserProfile` nie jest wersjonowany, a później M-01 wykazało brak kontroli w `FundXnt`.

Obecny kod poprawia `FundXnt`, ale sformułowanie „every instruction context” nadal wymaga pełnej, aktualnej macierzy instrukcja × konto.

Bez takiej tabeli lepsze jest:

```
Version checks are enforced for all versioned state accounts currently  
identified in the audited instruction contexts.  
UserProfile remains intentionally unversioned.
```

[DOC-02] R1-05 oznaczone jako w pełni naprawione, choć build bez sieci nadal istnieje

Tabela wskazuje:

```
test-periods safeguards log-only – fixed
```

- docs/SECURITY-AUDITS.md:115

H-01 rzeczywiście blokuje:

```
network-mainnet + test-periods
```

ale nie wymusza wyboru sieci.

Zatem bezpieczeństwo zależy częściowo od polityki release. Status może pozostać „fixed”, ale dowód powinien jawnie rozdzielić:

- twardy compile guard dla wariantu mainnet;
- proceduralne wykluczenie buildów bez feature'u sieci z deployu.

[DOC-03] „Evidence pipeline completed” jest nieudowodnione przez tę paczkę

Dokument stwierdza:

Evidence & release pipeline – completed

- docs/SECURITY-AUDITS.md:132-137

Kod pipeline'u jest poprawiony, lecz paczka nie zawiera poprawnego biegu aktualnego skryptu:

- TEST-LOG.txt jest stary i nieudany;
- TEST-LOG.sha256 nie istnieje.

Dlatego status powinien brzmieć:

Tooling fixes completed in source.
Fresh evidence run on commit ddf4b36 still must be attached to the audit package.

chyba że poprawny log istnieje w prywatnym repo, ale został omyłkowo pominięty w ZIP-ie.

[DOC-04] Historia przypisuje GPT bardziej pozytywny werdykt niż raport znajdujący się w tej rozmowie

Dokument podaje:

GPT verification: testnet / closed pilot ready

- docs/SECURITY-AUDITS.md:99

W poprzedniej odpowiedzi GPT werdykt brzmiał:

warunkowo gotowy do zamkniętego testnetu

i wymagał poprawy V-01–V-05.

Dokument dalej przyjmuje „stricter reading”, więc praktyczny rezultat jest poprawny, ale historyczny opis powinien używać dokładniejszego:

conditionally ready after the listed evidence-pipeline fixes

[DOC-05] Footnote z liniami może szybko się dezaktualizować

Tabela wielokrotnie podaje konkretne linie z wcześniejszego snapshotu.

- `docs/SECURITY-AUDITS.md:109-128`

Po formatowaniu lub zmianach dokumentacji line references mogą przestać być poprawne. Lepiej zapisywać:

- commit;
- ścieżkę;
- symbol/funkcję;
- opcjonalnie linie z danego commita.

Werdykt dokumentacyjny

Dokument jest wartościowy i generalnie uczciwy, ale wymaga korekty DOC-01–DOC-04, aby nie przedstawiał źródłowo wdrożonego pipeline'u jako dowodowo zakończonego bez aktualnego logu.

4. Powiązanie feature'u sieci z osobnym `declare_id!`

Obecny stan

Program nadal ma jeden placeholder:

```
declare_id!("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFsLnS");
```

- `programs/anl_staking/src/lib.rs:25-26`

`network-mainnet` i `network-testnet` służą obecnie tylko do wzajemnych blokad:

- `programs/anl_staking/src/lib.rs:11-15`

Nie determinują Program ID.

Rekomendacja

Warto zrobić przed testnetem

Nie jest to bezwzględnie konieczne dla poprawności kontraktu, ale mocno rekomenduję powiązanie feature'u sieci z Program ID **przed pierwszym testnetowym deployem**, nie dopiero przed mainnetem.

Przykład:

```
[cfg(feature = "network-mainnet")]
declare_id!("MAINNET_PROGRAM_ID");

[cfg(feature = "network-testnet")]
declare_id!("TESTNET_PROGRAM_ID");

[cfg(not(any(feature = "network-mainnet", feature = "network-testnet")))]
declare_id!("DEVELOPMENT_ONLY_PROGRAM_ID");
```

Jeszcze silniejszy wariant:

```
[cfg(not(any(feature = "network-mainnet", feature = "network-testnet")))]
compile_error!("network feature required for deployable program build");
```

ale może to utrudnić lokalne testy.

Rozsądny kompromis

Zostawić wariant bez feature'u dla `cargo test` i IDE, ale nadać mu jawny, niemożliwy do pomylenia development Program ID.

Wtedy:

- testnetowy artefakt ma testnetowy ID;
- mainnetowy artefakt ma mainnetowy ID;
- domyślny artefakt nie odpowiada żadnemu deployowanemu adresowi;
- hash i IDL są naturalnie związane z siecią;
- przypadkowy deploy domyślnego builda nie zastąpi właściwego programu.

Ocena pilności

Przed testnetem: zdecydowanie zalecane.

Przed immutable mainnetem: obowiązkowe.

Wczesne wdrożenie tej polityki pozwala przeciwzyć:

- osobne PDA;
- osobne IDL;
- osobne manifesty;
- osobne adresy w Anchor.toml;
- proces `anchor keys sync`;
- kontrolę zgodności binarki z adresem.

5. Kontrola regresji skryptów

`audit-evidence.sh`

Nie znaleziono obejścia przez `tee`

`pipefail` jest aktywny, więc błąd polecenia nie zostaje ukryty.

Nie znaleziono obejścia przez subshell `core`

Błąd któregośkolwiek `cargo test` w subshellu powinien zakończyć subshell kodem niezerowym, a następnie cały skrypt przez `set -e`.

`expect_fail` jest poprawne

Tymczasowe `set +e` obejmuje tylko przechwycenie oczekiwanego niepowodzenia. Następnie przywracane jest `set -e`, a kod i komunikat są sprawdzane.

Możliwe ulepszenia

1. Sprawdzić, czy `HEAD` nie zmienił się podczas długiego biegu:

```
test "$(git rev-parse HEAD)" = "$HEAD"
```

przed publikacją logu.

1. Sprawdzić, czy drzewo jest nadal czyste bezpośrednio przed `cp`.
2. Po publikacji policzyć także hash:
3. `Cargo.lock`;
4. Źródłowego archiwum;
5. opcjonalnie całego `git archive HEAD`.
6. Ustawić stabilne środowisko:

```
export CARGO_TERM_COLOR=never
export RUST_BACKTRACE=1
```

1. Zapisać:
2. `anchor --version`;
3. `solana --version`;
4. `cargo audit --version`;

5. `cargo deny --version`.

Skrypty build

Bramka Git

Nie widzę zwykłego obejścia staged/untracked. Poprzedni problem jest naprawiony.

Pozostałe ryzyka proceduralne

Skrypty nie gwarantują:

- że `anchor` jest oczekiwaną binarką;
- że toolchain jest niezmodyfikowany;
- że artefakt deployowany jest tym samym plikiem co zahashowany;
- że deployment nie omija skryptu;
- że lokalny system nie ma `assume-unchanged`.

Przed mainnetem release powinien powstawać w kontrolowanym CI, a deploy powinien pobierać dokładnie artefakt oznaczony hashem.

6. Status wcześniejszych ustaleń przedtestnetowych

Ustalenie	Status
V-01: supply-chain nieblokujący	Zamknięte w CI source
V-02: <code>audit-evidence.sh</code> nie fail-closed	Zamknięte w source
V-03: słabe clean-tree gates	Zamknięte
V-04: drugi guard bez sprawdzenia komunikatu	Zamknięte
V-05: README build policy i liczba 23	Zamknięte zasadniczo
Świeży evidence run związany z <code>ddf4b36</code>	Niezamknięte w dostarczonej paczce
Aktualny <code>TEST-LOG.sha256</code>	Brak w paczce
Oddzielny Program ID testnet	Niezamknięte
Porównanie IDL po rename handlerów	Niezamknięte
Bot zgodny z checkpoint API	Niezamknięte

7. Werdykt — zamknięty testnet

Kodowo i procesowo: prawie gotowy

Dowodowo dla tej konkretnej paczki: jeszcze warunkowo

Nie daję jeszcze całkowicie bezwarunkowego werdyktu, ponieważ paczka nie zawiera dwóch deklarowanych, kluczowych artefaktów:

1. udanego `TEST-LOG.txt` z HEAD `ddf4b36`;
2. odpowiadającego mu `TEST-LOG.sha256`.

Ponadto nadal istnieje placeholder Program ID.

Minimalne warunki do bezwarunkowego zamkniętego testnetu

1. wygenerować osobny testnetowy Program ID;
2. zaktualizować `declare_id!` i `Anchor.toml`;
3. najlepiej powiązać ID z `network-testnet`;
4. uruchomić aktualny `audit-evidence.sh` na czystym repo Git;
5. dołączyć świeże `TEST-LOG.txt` i `TEST-LOG.sha256`;
6. potwierdzić zielone cztery joby CI na tym samym HEAD;
7. zbudować przez `build-testnet.sh`;
8. zapisać i porównać SHA-256 wdrażanej binarki;
9. porównać IDL i discriminators po zmianach handlerów;
10. zaktualizować bota do nowego API;
11. zachować upgrade authority;
12. używać aktywów bez wartości albo z bardzo ograniczoną ekspozycją.

Po spełnieniu tych warunków:

zamknięty testnet — GOTOWY

Osiem aktualnych wyjątków RUSTSEC nie blokuje takiego testnetu, pod warunkiem że dotyczą wyłącznie zależności testowo-klienckich wskazanych w tym repo i nie są automatycznie akceptowane dla osobnego bota lub publicznej infrastruktury.

8. Aktualne blokery immutable mainnet

Bezwzględne

1. finalny mainnetowy Program ID;
2. osobny testnetowy i mainnetowy `declare_id!` lub równoważna twarda polityka;
3. świeży, udany evidence run związany z finalnym HEAD;
4. aktualny hash logu;
5. blocking CI na finalnym commicie;
6. upgrade Solana/Anchor i usunięcie wyjątków RUSTSEC;

7. property/fuzz tests księgowości XNT;
8. differential tests przeciwko `core`;
9. verifiable/reproducible build;
10. weryfikacja wdrożonej binarki;
11. hash finalnego SBF;
12. potwierdzenie zgodności Program ID, IDL i wszystkich PDA;
13. finalny audyt finalnego artefaktu release;
14. testnet observation period;
15. zaktualizowany i przetestowany bot;
16. formalna decyzja dotycząca upgrade authority;
17. brak `--final` przed zakończeniem całej checklisty.

Zalecane

- generowanie release wyłącznie w chronionym CI;
- podpisany manifest;
- SBOM;
- pin digestów GitHub Actions zamiast samych tagów;
- osobne review każdej zmiany w listach `ignore`;
- automatyczne przypomnienie przed datą przeglądu wyjątków;
- monitorowanie wygasłych, nierozliczonych shares;
- crank bounty lub redundantne crankery.

Ostateczny werdykt

Poprawki V-01–V-05 zostały wdrożone poprawnie w źródłach, a nowy `audit-evidence.sh` jest rzeczywiście fail-closed; nie znalazłem regresji związanej z `tee`, `pipefail` ani testami `core` w subshellu. Triage ośmiu advisory jest akceptowalny dla zamkniętego testnetu, ponieważ wskazane zależności występują w stosie dev/test, nie w programie SBF, choć wyjątki QUIC i kryptograficzne nie mogą zostać automatycznie rozszerzone na bota lub publiczną infrastrukturę.

Jednocześnie dostarczona paczka nie zawiera deklarowanego aktualnego dowodu wykonania: `TEST-LOG.txt` jest starym, nieudanym logiem z commita `27cd983`, a `TEST-LOG.sha256` nie istnieje. Z tego powodu zamknięty testnet oceniam jako **gotowy warunkowo po dołączeniu świeżego evidence run i nadaniu osobnego Program ID**, natomiast immutable mainnet pozostaje **niegotowy**.